

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 7 SORTING 1 (INSERTION, SELECTION, DAN
BUBBLE)



Disusun Oleh :

DIKA GIOWANDA

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, MEI 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan rahmat, hidayah, dan karunia-Nya sehingga laporan praktikum Struktur Data tentang algoritma sorting (*Insertion Sort*, *Selection Sort*, dan *Bubble Sort*) ini dapat diselesaikan dengan baik dan tepat waktu. Sholawat serta salam senantiasa tercurah kepada Nabi Muhammad SAW, keluarga, sahabat, dan seluruh pengikutnya hingga akhir zaman.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban, refleksi, serta dokumentasi atas kegiatan praktikum yang telah dilaksanakan, dengan fokus pada penerapan konsep struktur data khususnya algoritma pengurutan. *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* merupakan algoritma *sorting* dasar yang penting untuk dipahami karena menjadi fondasi dalam mempelajari algoritma pengurutan yang lebih kompleks. Dalam praktikum ini, penulis mempelajari cara kerja masing-masing algoritma, implementasinya dalam bahasa pemrograman, serta analisis kompleksitas waktu dan ruang dari setiap metode *sorting* tersebut.

Penyusunan laporan ini tidak mungkin terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih yang tulus kepada dosen pembimbing yang telah memberikan arahan, bimbingan, dan masukan yang berharga selama proses praktikum, serta kepada rekan-rekan mahasiswa yang turut berkontribusi dalam diskusi dan kolaborasi.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca, khususnya dalam memahami konsep dan implementasi algoritma sorting dalam struktur data.

Akhir kata, penulis berharap semoga ilmu yang diperoleh melalui praktikum ini dapat bermanfaat dan diterapkan dengan baik.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB 1 PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Tujuan Praktikum.....	1
1.3 Persyaratan Praktikum.....	1
1.4 Tempat dan Waktu Praktikum.....	2
1.5 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN PRAKTIKUM.....	3
2.1 Pengertian Pemrograman Sorting 1.....	3
2.3 Langkah-Langkah Pengerjaan Praktikum Pekan 7.....	4
BAB 3 PENUTUP.....	22
3.1 Kesimpulan.....	22
3.2 Saran.....	22
DAFTAR PUSTAKA.....	23
LAPORAN TUGAS.....	24

BAB 1

PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan konsep pemrograman Java. Kegiatan ini tidak hanya menekankan pada penguasaan teori, tetapi juga pada latihan penyusunan kode program, pengujian, hingga analisis hasil eksekusi. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual dengan kemampuan teknis pemrograman.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar pemrograman Struktur Data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Java.
3. Memahami tipe data dasar hingga kompleks, termasuk konsep penyimpanan data.
4. Mampu menerapkan struktur data linear (*Linked List, Stack, Queue*) dan dalam bahasa pemrograman
5. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
6. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
7. Mengetahui dan mengaplikasikan *ADT, ArrayList, Stack, Queue, Single Linked List, Double Linked List, Sorting 1 (Insertion, selection, bubble), Sorting 2 (shell, quick, merge), dan Searching (BFS dan DFS)*.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang *Java Development Kit (JDK)* dan *Integrated Development Environment (IDE)* atau *Eclips* yang direkomendasikan.
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

1.5 Manfaat Praktikum

Manfaat praktikum Struktur Data meliputi kemampuan memahami cara menyimpan data secara terstruktur, rapi, dan efisien, sehingga mengurangi penggunaan memori dan mempercepat akses data. Meningkatkan efisiensi penyimpanan dan manipulasi data, mengoptimalkan kecepatan algoritma, serta mempermudah penyelesaian masalah pemrograman yang kompleks. Selain itu, praktikum ini juga membantu membangun fondasi untuk mengembangkan aplikasi lintas platform yang lebih kompleks di berbagai bidang teknologi.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian Pemrograman Sorting 1

Sorting 1 adalah proses mengurutkan sekumpulan data secara teratur sesuai aturan tertentu, baik dari kecil ke besar (*ascending*) maupun sebaliknya (*descending*). Tujuannya adalah untuk mempermudah pencarian data (*searching*). Komponen utama algoritma *sorting* dalam praktikum struktur data berfokus pada iterasi (*pengulangan*), perbandingan nilai elemen, dan pertukaran/pergeseran posisi data.

Berikut adalah penjelasan tiga algoritma *sorting* dasar:

1. Insertion SortKonsep:

- Bekerja seperti saat Anda menyortir kartu di tangan. Algoritma ini membagi data menjadi dua bagian yaitu, bagian yang sudah terurut dan bagian yang belum terurut. Elemen dari bagian tak terurut akan diambil dan disisipkan ke posisi yang tepat pada bagian yang sudah terurut.
- Cara kerja: Dimulai dari elemen kedua, bandingkan dengan elemen di kirinya. Jika elemen kedua lebih kecil, geser elemen pertama ke kanan, lalu sisipkan elemen kedua di posisi pertama.
- Kelebihan: Sangat efisien untuk dataset kecil atau data yang sebagian sudah terurut.

2. Selection SortKonsep:

- Bekerja dengan cara memilih elemen paling kecil (atau paling besar) dari bagian data yang belum terurut, lalu menukarnya dengan elemen pertama dari bagian tersebut.
- Cara kerja: Cari nilai terkecil dari seluruh elemen. Tukar nilai tersebut dengan elemen pertama. Ulangi proses ini untuk sisa elemen (dari elemen kedua hingga selesai) sampai seluruh data terurut.

- Kelebihan: Mudah dipahami dan jumlah pertukarannya lebih sedikit dibandingkan *Bubble Sort*.

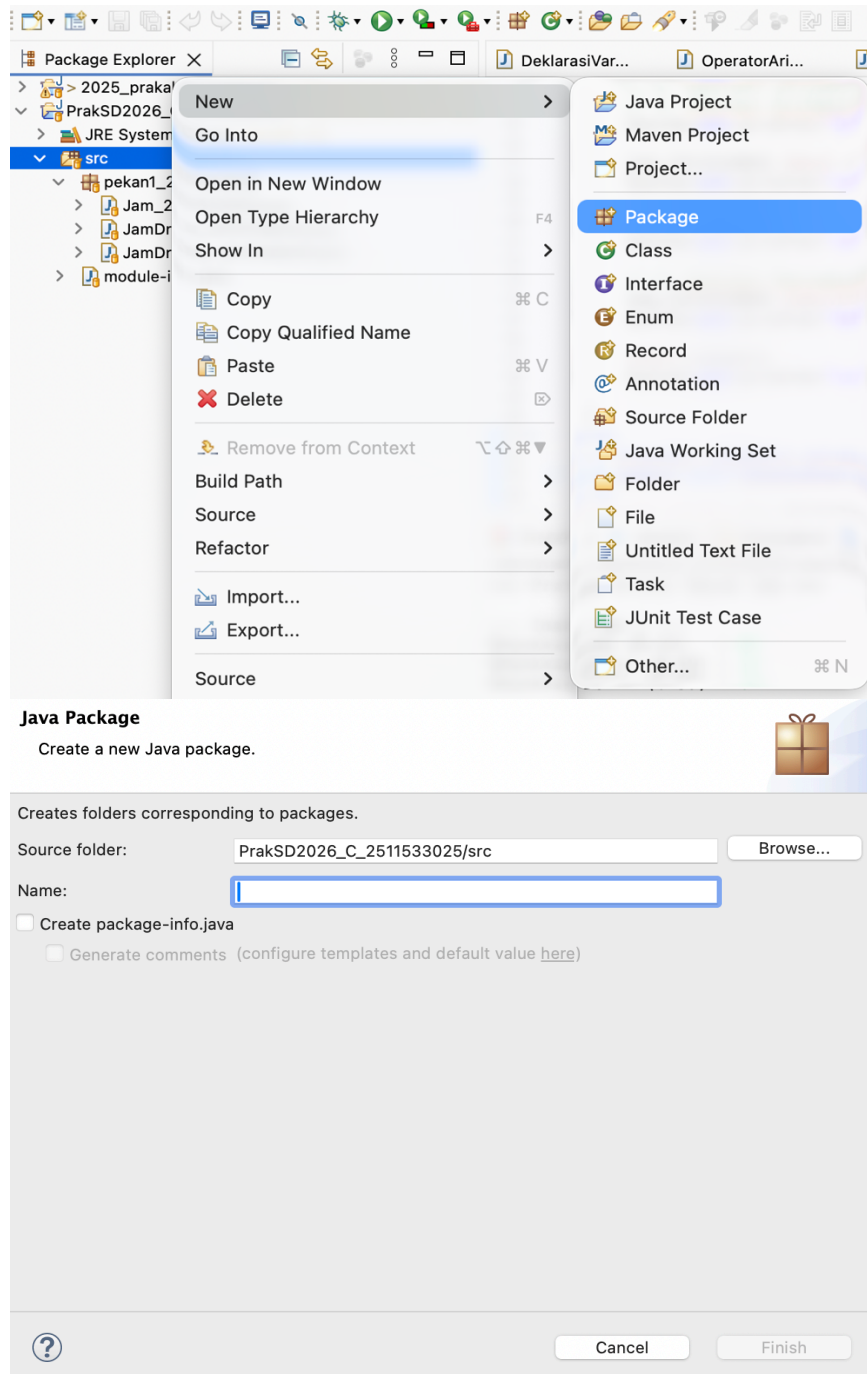
3. Bubble Sort Konsep:

- Mengurutkan data dengan cara membandingkan dua elemen yang bersebelahan secara terus-menerus dan menukarnya jika posisinya salah. Namanya diambil karena prosesnya seperti gelembung yang bergerak naik ke atas.
- Cara kerja: Bandingkan elemen ke-1 dan ke-2. Jika elemen ke-1 lebih besar, tukar posisinya. Lanjutkan ke elemen ke-2 dan ke-3, dan seterusnya hingga akhir *array*. Ulangi langkah tersebut sampai tidak ada lagi pertukaran data.
- Kelebihan: Algoritma paling simpel dan mudah dipahami, namun paling lambat jika digunakan untuk dataset yang besar.

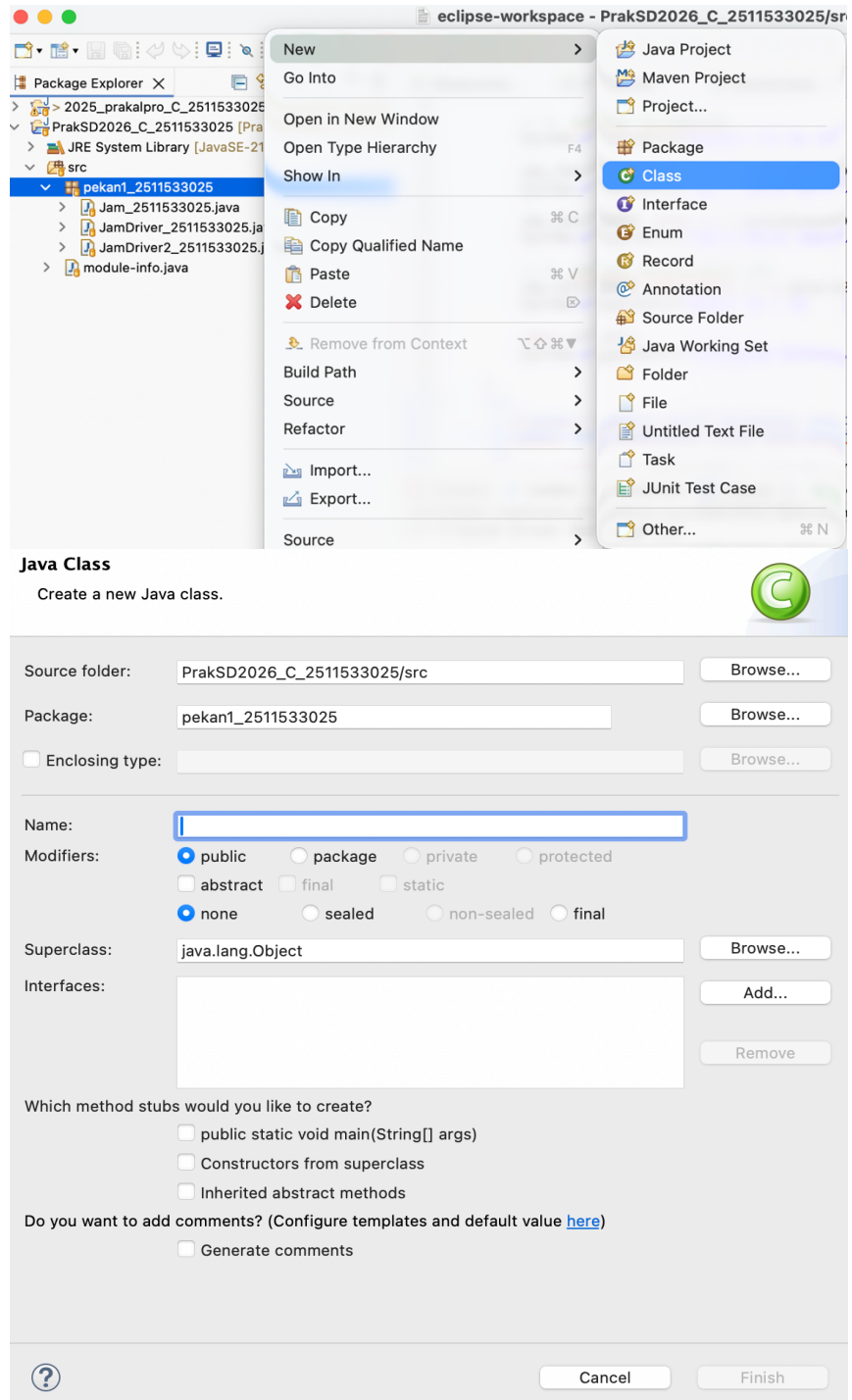
2.2 Langkah-Langkah Pengerjaan Praktikum Pekan 7

Sebelum ke langkah-langkah pengerjaan contoh proyek, saya akan menjelaskan terlebih dahulu gimana caranya menambahkan proyek nya terlebih dahulu :

1. Klik kanan pada folder src, pilih new, pilih package. Nanti akan muncul halaman Java Package, cukup masukan nama, nama wajib no spasi. Klik finish.



2. Klik kanan pada folder package, pilih new, pilih class. Nanti akan muncul halaman Java Class, cukup masukan nama class sesuai proyek yang akan dibuat, nama wajib no spasi. Klik finish.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru.

- A. Baiklah karena proses menambahkan proyek sudah dijelaskan, selanjutnya langkah-langkah pengerjaan proyek pekan 7:

a. Class InsertionSort_2511533025

1. Kode ini mengimplementasikan algoritma Insertion Sort untuk mengurutkan elemen dalam array bilangan bulat secara menaik (ascending). Cara kerjanya dimulai dengan mengambil satu elemen (disebut sebagai key) mulai dari indeks kedua, lalu membandingkannya dengan elemen-elemen di sebelah kirinya yang sudah dianggap terurut. Selama elemen di sebelah kiri lebih besar dari key, kode akan menggeser elemen-elemen tersebut satu posisi ke kanan untuk menciptakan ruang. Setelah menemukan posisi yang tepat atau mencapai awal array, key tersebut disisipkan ke dalam posisi yang kosong, dan proses ini diulangi hingga seluruh elemen dalam array berada dalam urutan yang benar.

```
DeklarasiVar... QueueArray_2... MusikDriver... InsertionSor... X SelectionSor...
1 package Pekan7_2511533025;
2
3 public class InsertionSort_2511533025 {
4     public static void insertionSort(int[] arr) {
5         int n_3025 = arr.length;
6         for (int i_3025 = 1; i_3025 < n_3025; i_3025++) {
7             int key_3025 = arr[i_3025];
8             int j_3025 = i_3025 - 1;
9             while (j_3025 >= 0 && arr[j_3025] > key_3025) {
10                 arr[j_3025 + 1] = arr[j_3025];
11                 j_3025--;
12             }
13             arr[j_3025 + 1] = key_3025;
14         }
15     }
```

2. Kode ini untuk mendemonstrasikan proses pengurutan data pada sebuah array menggunakan metode Insertion Sort. Program dimulai dengan menginisialisasi sebuah array berisi angka acak, kemudian menampilkan isi array tersebut ke layar dalam kondisi belum terurut. Setelah itu, program memanggil fungsi insertionSort(arr) untuk mengurutkan elemen-elemen di dalamnya secara menaik (ascending), dan diakhiri dengan mencetak kembali isi array yang telah terurut.

```
16 public static void main(String[] args) {
17     int arr[] = {23, 78, 45, 8, 32, 56, 1};
18     int n_3025 = arr.length;
19     System.out.printf("array yang belum terurut:\n");
20     for (int i_3025 = 0; i_3025 < n_3025; i_3025++)
21         System.out.print(arr[i_3025] + " ");
22     System.out.println("");
23     insertionSort(arr);
24     System.out.printf("array yang terurut:\n");
25     for (int i_3025 = 0; i_3025 < n_3025; i_3025++)
26         System.out.print(arr[i_3025] + " ");
27     System.out.println("");
28 }
29
30 }
```

b. Class SelectionSort_2511533025

1. Kode ini untuk mengimplementasikan algoritma Selection Sort dalam bahasa Java untuk mengurutkan sebuah array bilangan bulat (arr) secara menaik (ascending). Logikanya bekerja dengan cara mencari elemen terkecil dari bagian array yang belum terurut melalui perulangan dalam (inner loop) dan menyimpan indeksinya ke dalam variabel minIndex_3025. Setelah elemen terkecil pada iterasi tersebut ditemukan, kode akan melakukan pertukaran posisi (swap) menggunakan variabel bantuan temp_3025 untuk menempatkan elemen terkecil tersebut ke posisi yang benar di bagian depan array. Proses ini diulangi terus-menerus oleh perulangan luar (outer loop) hingga seluruh elemen di dalam array terurut dengan benar.

```
1 package Pekan7_2511533025;
2
3 public class SelectionSort_2511533025 {
4     public static void selectionSort(int[] arr) {
5         int n_3025 = arr.length;
6         for (int i_3025 = 0; i_3025 < n_3025; i_3025++) {
7             int minIndex_3025 = i_3025;
8             for (int j_3025 = i_3025 + 1; j_3025 < n_3025; j_3025++) {
9                 if (arr[j_3025] < arr[minIndex_3025]) {
10                     minIndex_3025 = j_3025;
11                 }
12             }
13             int temp_3025 = arr[i_3025];
14             arr[i_3025] = arr[minIndex_3025];
15             arr[minIndex_3025] = temp_3025;
16         }
17     }
18 }
```

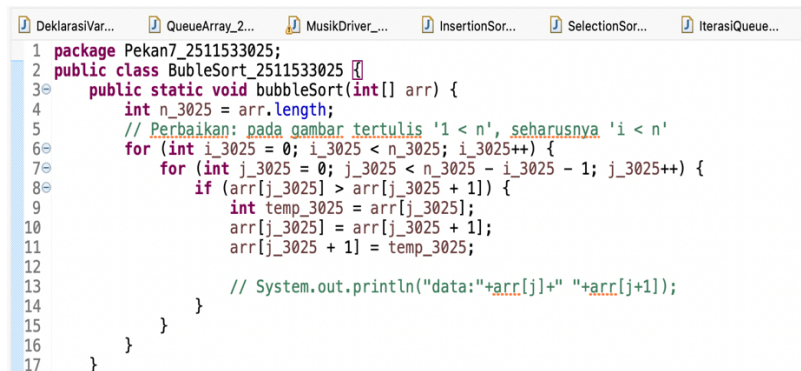
2. Kode ini untuk mendemonstrasikan proses pengurutan data dalam sebuah array menggunakan metode selection sort. Program dimulai dengan menginisialisasi sebuah array bilangan bulat, kemudian menampilkan isi array tersebut ke layar dalam kondisi belum terurut, memanggil metode selectionSort(arr) untuk menyusun elemen-elemennya dari nilai terkecil ke terbesar, dan diakhiri dengan mencetak kembali isi array yang kini telah dalam keadaan terurut.

```
18 public static void main(String[] args) {
19     int arr[] = {23, 78, 45, 8, 32, 56, 1};
20     int n_3025 = arr.length;
21     System.out.printf("array yang belum terurut:\n");
22     for (int i_3025 = 0; i_3025 < n_3025; i_3025++)
23         System.out.print(arr[i_3025] + " ");
24     System.out.println("");
25     selectionSort(arr);
26     System.out.printf("array yang terurut:\n");
27     for (int i_3025 = 0; i_3025 < n_3025; i_3025++)
28         System.out.print(arr[i_3025] + " ");
29     System.out.println("");
30 }
31 }
```

c. Class BubleSort_2511533025

1. Kode ini untuk implementasi algoritma Bubble Sort dalam bahasa pemrograman Java yang berfungsi untuk mengurutkan elemen dalam sebuah array integer secara menaik (ascending). Logika utama kode ini menggunakan dua lapis perulangan

(nested loop) yaitu, perulangan luar mengontrol jumlah iterasi secara keseluruhan, sementara perulangan dalam bertugas membandingkan dua elemen yang letaknya bersebelahan. Jika elemen di sisi kiri memiliki nilai yang lebih besar daripada elemen di sisi kanan ($\text{arr}[\text{j_3025}] > \text{arr}[\text{j_3025} + 1]$), maka posisi keduanya akan ditukar menggunakan variabel bantuan `temp_3025`. Proses perbandingan dan pertukaran ini dilakukan berulang kali.

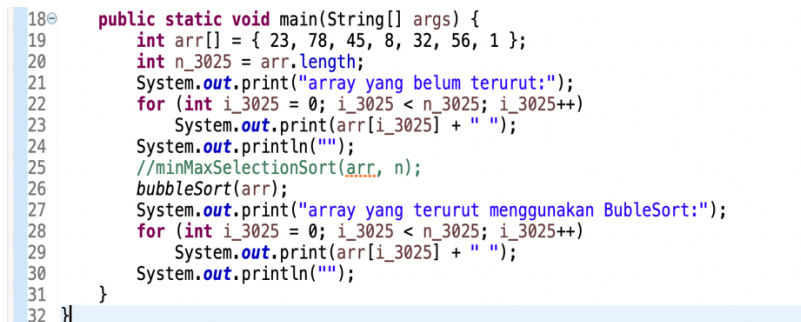


```

1 package Pekan7_2511533025;
2 public class BubleSort_2511533025 {
3     public static void bubbleSort(int[] arr) {
4         int n_3025 = arr.length;
5         // Perbaikan: pada gambar tertulis '1 < n', seharusnya 'i < n'
6         for (int i_3025 = 0; i_3025 < n_3025; i_3025++) {
7             for (int j_3025 = 0; j_3025 < n_3025 - i_3025 - 1; j_3025++) {
8                 if (arr[j_3025] > arr[j_3025 + 1]) {
9                     int temp_3025 = arr[j_3025];
10                    arr[j_3025] = arr[j_3025 + 1];
11                    arr[j_3025 + 1] = temp_3025;
12                }
13                // System.out.println("data:"+arr[j]+" "+arr[j+1]);
14            }
15        }
16    }
17 }

```

2. Kode ini untuk mendemonstrasikan proses pengurutan data pada sebuah array menggunakan algoritma Bubble Sort. Program dimulai dengan menginisialisasi sebuah array bilangan bulat dan mencetak isi array tersebut dalam kondisi belum terurut ke layar. Selanjutnya, kode memanggil fungsi `bubbleSort(arr)` untuk menyusun elemen-elemen array secara menaik (ascending), lalu diakhiri dengan mencetak.



```

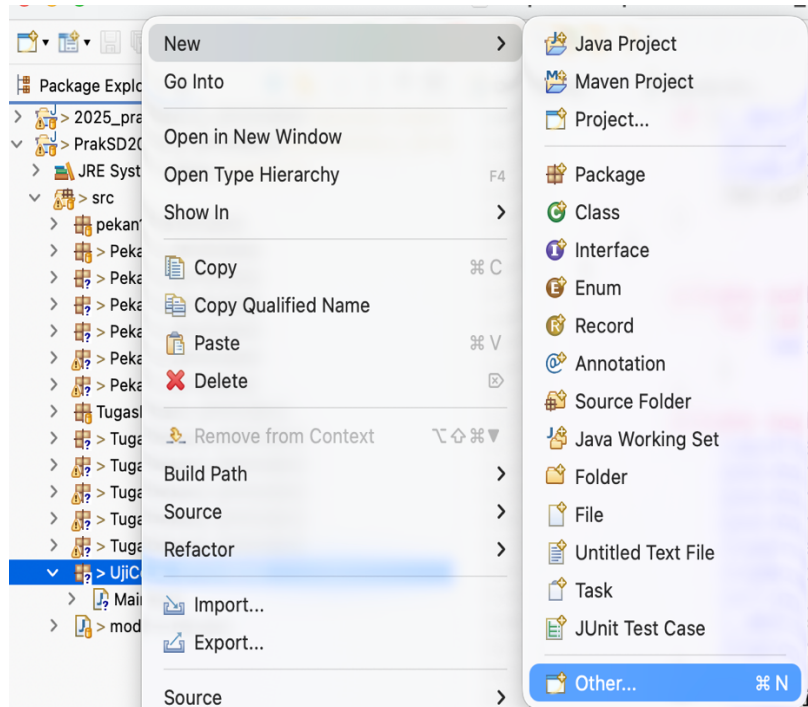
18 public static void main(String[] args) {
19     int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
20     int n_3025 = arr.length;
21     System.out.print("array yang belum terurut:");
22     for (int i_3025 = 0; i_3025 < n_3025; i_3025++)
23         System.out.print(arr[i_3025] + " ");
24     System.out.println("");
25     //minMaxSelectionSort(arr, n);
26     bubbleSort(arr);
27     System.out.print("array yang terurut menggunakan BubleSort:");
28     for (int i_3025 = 0; i_3025 < n_3025; i_3025++)
29         System.out.print(arr[i_3025] + " ");
30     System.out.println("");
31 }
32 }

```

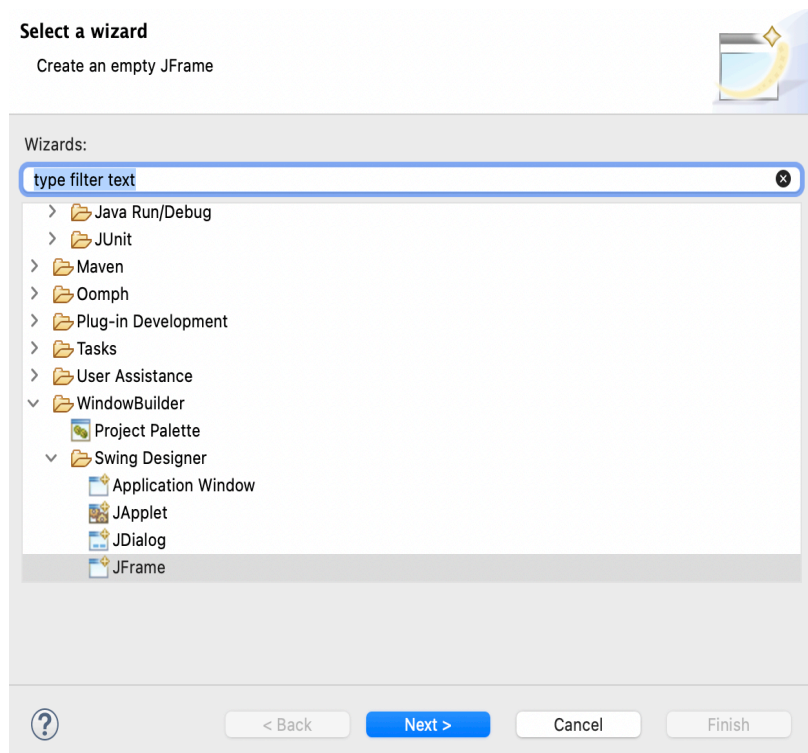
d. Class InsertionGUI_2511533025

Class yang ini berbeda dengan class sebelumnya karena tampilan outputnya berbentuk GUI.

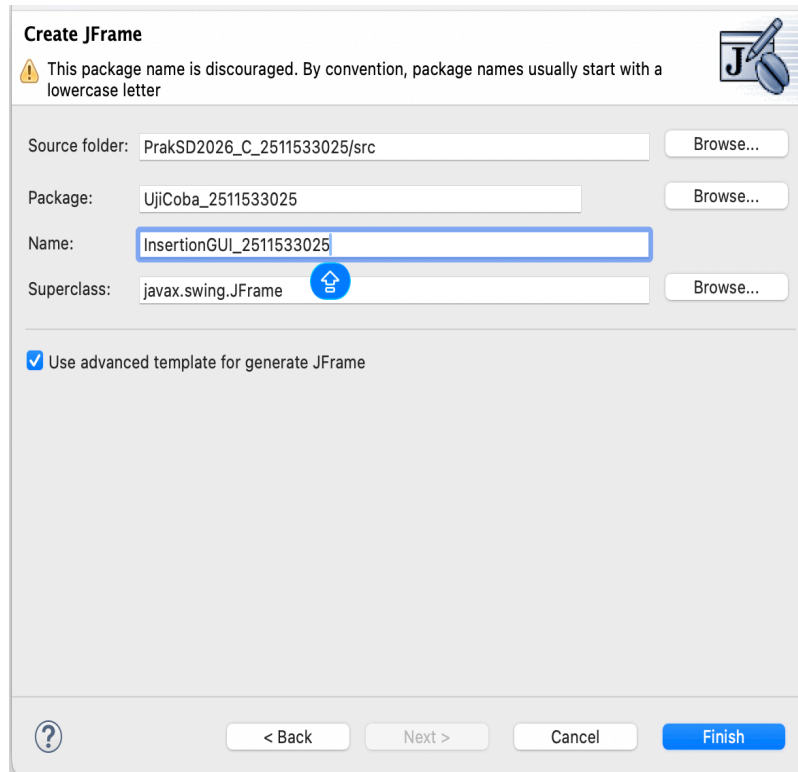
1. Buat class terlebih dahulu, caranya klik kanan pada package yang ingin digunakan, pilih new, terus pilih Other.



2. Kemudian pilih WindowBuilder, lalu pilih Swing Designer, terus pilih JFrame. Klik Next.



3. Terakhir masukan nama pada bagian name, klik Finish. Lanjut masukan kode sesuai program yang ingin dibuat.



4. Kode ini untuk menyiapkan komponen-komponen antarmuka grafis (GUI). Dengan mengimpor berbagai kelas dari package java.awt dan javax.swing.

```

1 package Pekan7_2511533025;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.EventQueue;
7 import java.awt.FlowLayout;
8 import java.awt.Font;
9
10 import javax.swing.BorderFactory;
11 import javax.swing.JButton;
12 import javax.swing.JFrame;
13 import javax.swing.JLabel;
14 import javax.swing.JOptionPane;
15 import javax.swing.JPanel;
16 import javax.swing.JScrollPane;
17 import javax.swing.JTextArea;
18 import javax.swing.JTextField;
19 import javax.swing.SwingConstants;
20 import javax.swing.border.EmptyBorder;
21

```

5. Kode ini mendefinisikan JFrame untuk membangun antarmuka grafis (GUI) yang berfungsi memvisualisasikan proses algoritma insertion sort. Di dalamnya terdapat berbagai atribut privat yang mencakup komponen Swing seperti JLabel dan JPanel untuk visualisasi elemen array, tombol kontrol (step, reset, set) untuk interaksi pengguna, serta JTextArea untuk menampilkan log langkah-langkah pengurutan. Selain komponen visual, kode ini juga mendefinisikan variabel logika utama seperti array data, penunjuk indeks (i dan j) untuk

algoritma pengurutan, serta status sorting untuk melacak jalannya simulasi pengurutan secara bertahap.

```
22 public class InsertionGUI_2511533025 extends JFrame {
23
24     private static final long serialVersionUID = 1L;
25     private int[] array_3025;
26     private JLabel[] labelArray_3025;
27     private JButton stepButton_3025, resetButton_3025, setButton_3025;
28     private JTextField inputField_3025;
29     private JPanel panelArray_3025;
30     private JTextArea stepArea_3025;
31
32     private int i_3025 = 1, j_3025;
33     private boolean sorting_3025 = false;
34     private int stepCount_3025 = 1;
35 }
```

6. Kode ini untuk menginisialisasi properti dasar dari jendela antarmuka grafis (GUI) berbasis Java Swing. Di dalamnya, program mengatur judul jendela menjadi "Insertion Sort Langkah per Langkah", menentukan ukuran jendela sebesar 750x400 piksel, dan memastikan aplikasi berhenti sepenuhnya saat jendela ditutup melalui perintah EXIT_ON_CLOSE. Selain itu, kode ini menyertakan fungsi setLocationRelativeTo(null) untuk memposisikan jendela tepat di tengah layar saat dijalankan, serta menetapkan BorderLayout sebagai manajer tata letak guna mengatur posisi komponen visual di dalam frame tersebut.

```
37 /**
38  * Create the frame.
39  */
40 public InsertionGUI_2511533025() {
41     setTitle("Insertion Sort Langkah per Langkah");
42     setSize(750, 400);
43     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
44     setLocationRelativeTo(null);
45     setLayout(new BorderLayout());
46 }
```

7. Kode ini untuk membangun antarmuka pengguna (GUI) berbasis Java Swing berupa panel input yang menggunakan FlowLayout. Kode ini menginisialisasi sebuah JPanel bernama inputPanel_3025 yang diisi dengan tiga komponen utama: sebuah label instruksi (JLabel) yang meminta pengguna memasukkan angka dengan pemisah koma, sebuah kotak teks (JTextField) dengan lebar 30 kolom untuk menerima input tersebut, serta sebuah tombol (JButton) berlabel "Set Array" yang kemungkinan digunakan untuk memproses data yang telah dimasukkan.

```
47 // Panel input
48 JPanel inputPanel_3025 = new JPanel(new FlowLayout());
49 inputField_3025 = new JTextField(30);
50 setButton_3025 = new JButton("Set Array");
51 inputPanel_3025.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
52 inputPanel_3025.add(inputField_3025);
53 inputPanel_3025.add(setButton_3025);
54 }
```

8. Kode ini untuk menginisialisasi dan mengatur tata letak sebuah objek penampung (kontainer) bernama `panelArray_3025` menggunakan pustaka Swing dalam bahasa pemrograman Java. Pada baris pertama, sebuah objek baru dari kelas `JPanel` dibuat, yang berperan sebagai kanvas atau area untuk menampung komponen antarmuka pengguna (UI) lainnya. Kemudian, baris berikutnya menetapkan manajer tata letak komponen tersebut menjadi `FlowLayout`.

```
55 // Panel array visual
56 panelArray_3025 = new JPanel();
57 panelArray_3025.setLayout(new FlowLayout());
58
```

9. Kode ini untuk menyiapkan panel kontrol pada antarmuka pengguna (GUI) menggunakan Java Swing dengan membuat sebuah kontainer `JPanel` dan dua buah tombol (`JButton`) yang masing-masing diberi label "Langkah Selanjutnya" dan "Reset". Melalui baris kode tersebut, tombol "Langkah Selanjutnya" diatur agar berada dalam kondisi tidak aktif secara default menggunakan fungsi `setEnabled(false)`, yang berarti tombol tersebut tidak dapat diklik oleh pengguna saat aplikasi pertama kali dijalankan. Akhirnya, kedua tombol tersebut dimasukkan ke dalam `controlPanel_3025` menggunakan metode `add` agar tersusun secara berkelompok di dalam satu wadah panel yang sama.

```
59 // Panel kontrol
60 JPanel controlPanel_3025 = new JPanel();
61 stepButton_3025 = new JButton("Langkah Selanjutnya");
62 resetButton_3025 = new JButton("Reset");
63 stepButton_3025.setEnabled(false);
64 controlPanel_3025.add(stepButton_3025);
65 controlPanel_3025.add(resetButton_3025);
```

10. Kode ini berfungsi untuk membuat dan mengonfigurasi sebuah area teks (`JTextArea`) yang digunakan sebagai tampilan log langkah-langkah. Objek `stepArea_3025` diinisialisasi dengan ukuran 8 baris dan 60 kolom, diatur agar tidak dapat diedit (`setEditable(false)`) oleh pengguna, serta menggunakan jenis font "Monospaced" berukuran 14. Terakhir, area teks tersebut dimasukkan ke dalam sebuah `JScrollPane`.

```
66 // Area teks untuk log langkah-langkah
67 stepArea_3025 = new JTextArea(8, 60);
68 stepArea_3025.setEditable(false);
69 stepArea_3025.setFont(new Font("Monospaced", Font.PLAIN, 14));
70 JScrollPane scrollPane_3025 = new JScrollPane(stepArea_3025);
71
```

11. Kode ini untuk menyusun tata letak antarmuka grafis (GUI) berbasis Java Swing dengan menambahkan empat komponen berbeda ke dalam sebuah `JFrame` atau kontainer utama menggunakan sistem `BorderLayout`. Secara spesifik, baris kode tersebut menempatkan `inputPanel_3025` di bagian atas

(NORTH), panelArray_3025 di area tengah sebagai konten utama (CENTER), controlPanel_3025 di bagian bawah untuk navigasi atau tombol aksi (SOUTH), serta scrollPane_3025 di sisi kanan frame (EAST). Penggunaan BorderLayout ini memungkinkan setiap komponen menyesuaikan ukurannya secara otomatis.

```
72 // Tambahkan panel ke frame
73 add(inputPanel_3025, BorderLayout.NORTH);
74 add(panelArray_3025, BorderLayout.CENTER);
75 add(controlPanel_3025, BorderLayout.SOUTH);
76 add(scrollPane_3025, BorderLayout.EAST);
77
```

12. Kode ini untuk menambahkan penanganan kejadian (event listener) pada tiga tombol antarmuka pengguna (GUI) menggunakan ekspresi lambda dalam bahasa pemrograman Java. Tombol setButton_3025 dikonfigurasi untuk memicu fungsi setArrayFromInput() guna memproses data input ke dalam array, sementara stepButton_3025 menjalankan fungsi performStep() untuk mengeksekusi langkah logika berikutnya dalam program. Terakhir, resetButton_3025 diprogram untuk memanggil metode reset() yang akan mengembalikan kondisi.

```
78 // Event Set Array
79 setButton_3025.addActionListener(e -> setArrayFromInput());
80
81 // Event Langkah Selanjutnya
82 stepButton_3025.addActionListener(e -> performStep());
83
84 // Event Reset
85 resetButton_3025.addActionListener(e -> reset());
86 }
```

13. Kode fungsi setArrayFromInput() ini bertujuan untuk mengambil input teks dari komponen inputField_3025, membersihkan spasi di awal dan akhir teks, lalu mengonversinya menjadi sebuah array integer bernama array_3025. Secara teknis, kode ini memecah string input berdasarkan tanda koma menggunakan metode split(","), menentukan ukuran array sesuai jumlah potongan teks yang didapat, dan menggunakan perulangan for untuk mengubah setiap potongan string tersebut menjadi angka melalui Integer.parseInt(). Kode ini juga dilengkapi dengan mekanisme penanganan kesalahan menggunakan blok try-catch; jika pengguna memasukkan karakter yang bukan angka, program akan menangkap NumberFormatException dan menampilkan pesan peringatan melalui JOptionPane yang meminta pengguna untuk hanya memasukkan angka yang dipisahkan dengan koma.

```

87 private void setArrayFromInput() {
88     String text_3025 = inputField_3025.getText().trim();
89     if (text_3025.isEmpty()) return;
90     String[] parts = text_3025.split(",");
91     array_3025 = new int[parts.length];
92     try {
93         for (int k_3025 = 0; k_3025 < parts.length; k_3025++) {
94             array_3025[k_3025] = Integer.parseInt(parts[k_3025].trim()); }
95     } catch (NumberFormatException e) {
96         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
97             + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
98         return; }

```

14. Kode ini untuk menginisialisasi ulang tampilan visual sebuah array ke dalam panel antarmuka (GUI) menggunakan pustaka Swing. Secara teknis, kode ini menyetel ulang variabel kontrol (seperti indeks dan status pengurutan), mengosongkan isi panelArray_3025, lalu membuat array baru berisi objek JLabel yang masing-masing merepresentasikan elemen dari data array_3025. Di dalam perulangan for, setiap label dikonfigurasi dengan jenis font Arial tebal berukuran 24, diberi garis tepi hitam, diatur ukurannya menjadi (50 x 50) piksel, diposisikan teksnya ke tengah, dan akhirnya ditambahkan ke dalam panel. Proses ini diakhiri dengan perintah `revalidate()` dan `repaint()` untuk memastikan perubahan tampilan langsung diterapkan.

```

99     i_3025 = 1;
100     stepCount_3025 = 1;
101     sorting_3025 = true;
102     stepButton_3025.setEnabled(true);
103     stepArea_3025.setText("");
104     panelArray_3025.removeAll();
105     labelArray_3025 = new JLabel[array_3025.length];
106     for (int k_3025 = 0; k_3025 < array_3025.length; k_3025++) {
107         labelArray_3025[k_3025] = new JLabel(String.valueOf(array_3025[k_3025]));
108         labelArray_3025[k_3025].setFont(new Font("Arial", Font.BOLD, 24));
109         labelArray_3025[k_3025].setBorder(BorderFactory.createLineBorder(Color.BLACK));
110         labelArray_3025[k_3025].setPreferredSize(new Dimension(50, 50));
111         labelArray_3025[k_3025].setHorizontalAlignment(SwingConstants.CENTER);
112         panelArray_3025.add(labelArray_3025[k_3025]);
113     }
114     panelArray_3025.revalidate();
115     panelArray_3025.repaint();
116 }

```

15. Kode ini mendefinisikan fungsi `performStep()` yang mengimplementasikan satu langkah perulangan dari algoritma Insertion Sort secara interaktif untuk kebutuhan visualisasi. Fungsi ini bekerja dengan mengambil satu elemen (`key_3025`) dari indeks saat ini, lalu membandingkan dan menggeser elemen-elemen di sisi kirinya yang lebih besar ke arah kanan melalui perulangan `while` agar elemen tersebut dapat disisipkan pada posisi yang tepat. Selain melakukan manipulasi data pada array, kode ini juga mencatat setiap langkah proses ke dalam log (`stepLog_3025`), memperbarui tampilan antarmuka (UI), dan meningkatkan penghitung langkah. Jika indeks telah mencapai akhir panjang array, fungsi akan menghentikan status pengurutan.

```

117 private void performStep() {
118     if (i_3025 < array_3025.length && sorting_3025) {
119         int key_3025 = array_3025[i_3025];
120         j_3025 = i_3025 - 1;
121
122         StringBuilder stepLog_3025 = new StringBuilder();
123         stepLog_3025.append("Langkah ").append(stepCount_3025).
124         append(": Memasukkan ").append(key_3025).append("\n");
125
126         while (j_3025 >= 0 && array_3025[j_3025] > key_3025) {
127             array_3025[j_3025 + 1] = array_3025[j_3025];
128             j_3025--;
129         }
130         array_3025[j_3025 + 1] = key_3025;
131
132         updateLabels();
133         stepLog_3025.append("Hasil: ").append(arrayToString(array_3025)).append("\n\n");
134         stepArea_3025.append(stepLog_3025.toString());
135
136         i_3025++;
137         stepCount_3025++;
138
139         if (i_3025 == array_3025.length) {
140             sorting_3025 = false;
141             stepButton_3025.setEnabled(false);
142             JOptionPane.showMessageDialog(this, "Sorting selesai!");
143         }
144     }
145 }

```

16. Kode ini untuk memperbarui teks pada sekumpulan objek label (kemungkinan besar komponen antarmuka pengguna seperti JLabel di Java) secara otomatis dengan melakukan iterasi melalui sebuah perulangan for. Kode ini menggunakan variabel indeks k_3025 untuk mengakses setiap elemen dalam array data bernama array_3025 dan secara bersamaan memetakan nilai tersebut ke elemen yang sesuai dalam array label labelArray_3025. Di dalam perulangan, nilai dari array_3025 dikonversi menjadi tipe data String menggunakan perintah String.valueOf() sebelum akhirnya ditetapkan sebagai teks tampilan label melalui metode .setText().

```

146 private void updateLabels() {
147     for (int k_3025 = 0; k_3025 < array_3025.length; k_3025++) {
148         labelArray_3025[k_3025].setText(String.valueOf(array_3025[k_3025]));
149     }
150 }
151

```

17. Kode ini berfungsi untuk mengembalikan status aplikasi ke kondisi awal dengan membersihkan antarmuka pengguna dan mengatur ulang variabel kontrol. Kode ini mengosongkan teks pada inputField dan stepArea, menghapus semua komponen dari panelArray sekaligus memperbarui tampilannya (revalidate dan repaint), serta menonaktifkan tombol stepButton. Selain itu, fungsi ini mengatur ulang variabel logika sorting menjadi false dan mengembalikan nilai penghitung i serta stepCount ke angka 1, sehingga program siap untuk memulai proses atau simulasi baru dari awal.

```

152 private void reset() {
153     inputField_3025.setText("");
154     panelArray_3025.removeAll();
155     panelArray_3025.revalidate();
156     panelArray_3025.repaint();
157     stepArea_3025.setText("");
158     stepButton_3025.setEnabled(false);
159     sorting_3025 = false;
160     i_3025 = 1;
161     stepCount_3025 = 1;
162 }
163

```

18. Kode ini fungsi `arrayToString` ini bertujuan untuk mengubah sebuah array bilangan bulat (`int[]`) menjadi satu string tunggal yang rapi, di mana setiap elemennya dipisahkan oleh tanda koma dan spasi. Kode ini menggunakan objek `StringBuilder` untuk membangun string secara efisien melalui perulangan `for` yang menyusuri seluruh isi array. Di dalam perulangan, setiap angka ditambahkan ke dalam urutan string, dan terdapat pengecekan kondisi (`if`) untuk memastikan bahwa tanda pemisah (", ") hanya ditambahkan di antara elemen-elemen saja dan tidak muncul di akhir setelah elemen terakhir. Hasil akhirnya kemudian dikonversi menjadi tipe data `String` dan dikembalikan sebagai keluaran fungsi tersebut.

```

164 private String arrayToString(int[] arr) {
165     StringBuilder sb_3025 = new StringBuilder();
166     for (int k_3025 = 0; k_3025 < arr.length; k_3025++) {
167         sb_3025.append(arr[k_3025]);
168         if (k_3025 < arr.length - 1) sb_3025.append(", ");
169     }
170     return sb_3025.toString();
171 }
172
173

```

19. Kode ini berfungsi sebagai titik awal eksekusi aplikasi GUI berbasis Swing. Kode ini menggunakan metode `SwingUtilities.invokeLater` dengan ekspresi lambda untuk memastikan bahwa pembuatan dan penampilan antarmuka pengguna (`InsertionGUI_2511533025`) dilakukan pada Event Dispatch Thread (EDT), yang merupakan praktik terbaik demi menjaga keamanan thread dan mencegah terjadinya freeze atau ketidakstabilan pada antarmuka. Di dalamnya, objek gui dibuat dari kelas tersebut dan kemudian diperintahkan untuk tampil di layar melalui instruksi `gui.setVisible(true)`.

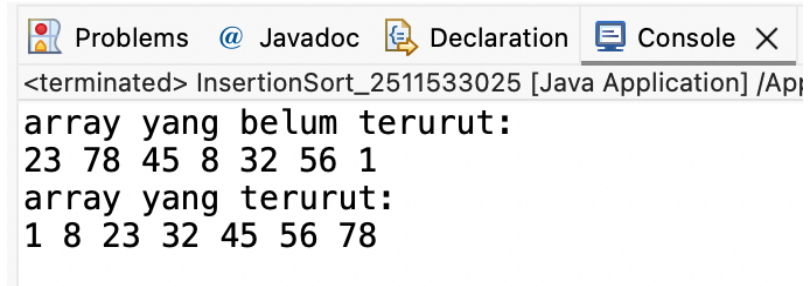
```

171 public static void main(String[] args) {
172     SwingUtilities.invokeLater(() -> {
173         InsertionGUI_2511533025 gui = new InsertionGUI_2511533025();
174         gui.setVisible(true);
175     });
176 }
177

```

f. Hasil projek pekan 7.

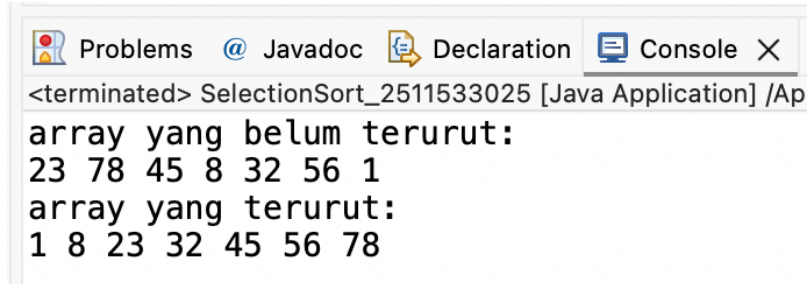
1. Output dari Class `InsertionSort_2511533025`.



The screenshot shows the Eclipse IDE's console window. The title bar includes 'Problems', 'Javadoc', 'Declaration', and 'Console'. The console text shows the execution of 'InsertionSort_2511533025 [Java Application] /Ap'. The output consists of two lines of text: 'array yang belum terurut:' followed by the numbers '23 78 45 8 32 56 1', and 'array yang terurut:' followed by the numbers '1 8 23 32 45 56 78'.

```
<terminated> InsertionSort_2511533025 [Java Application] /Ap  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

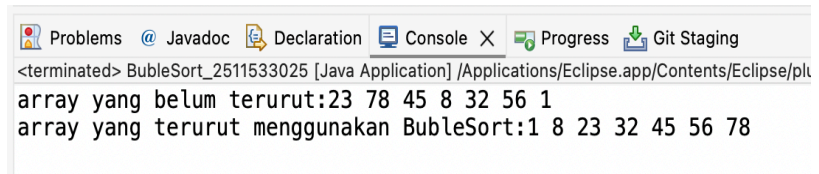
2. Output dari Class SelectionSort_2511533025.



The screenshot shows the Eclipse IDE's console window. The title bar includes 'Problems', 'Javadoc', 'Declaration', and 'Console'. The console text shows the execution of 'SelectionSort_2511533025 [Java Application] /Ap'. The output is identical to the previous screenshot: 'array yang belum terurut:' followed by '23 78 45 8 32 56 1', and 'array yang terurut:' followed by '1 8 23 32 45 56 78'.

```
<terminated> SelectionSort_2511533025 [Java Application] /Ap  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

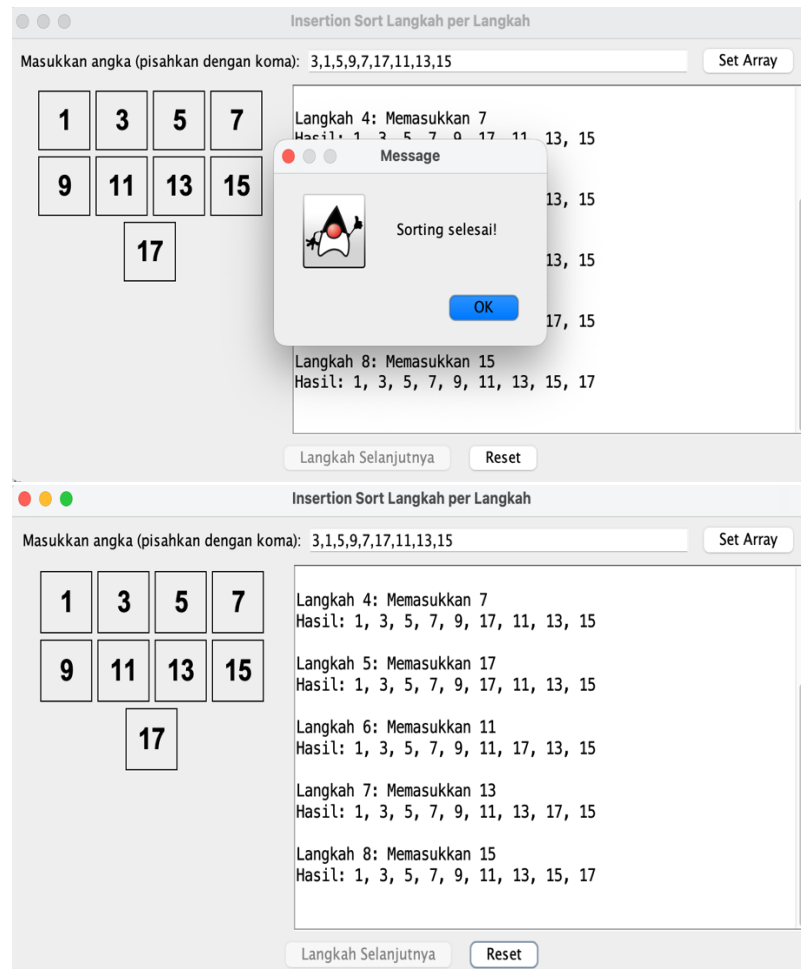
3. Output dari Class BubleSort_2511533025.



The screenshot shows the Eclipse IDE's console window. The title bar includes 'Problems', 'Javadoc', 'Declaration', 'Console', 'Progress', and 'Git Staging'. The console text shows the execution of 'BubleSort_2511533025 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/pl'. The output shows 'array yang belum terurut:23 78 45 8 32 56 1' and 'array yang terurut menggunakan BubleSort:1 8 23 32 45 56 78'.

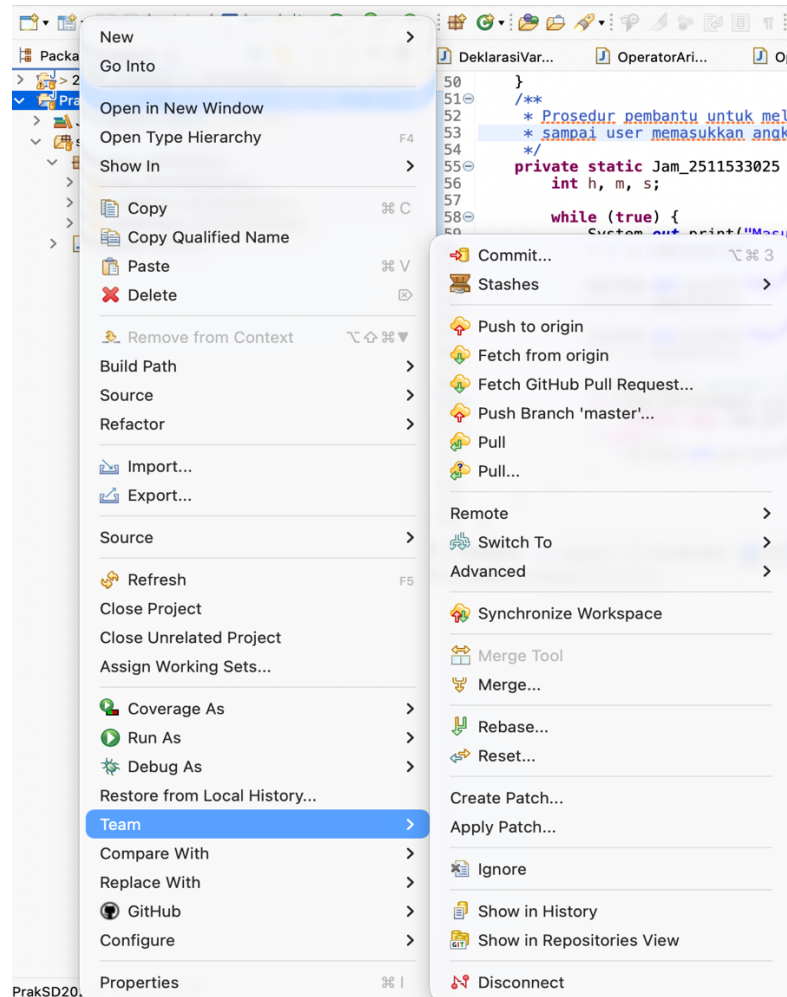
```
<terminated> BubleSort_2511533025 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/pl  
array yang belum terurut:23 78 45 8 32 56 1  
array yang terurut menggunakan BubleSort:1 8 23 32 45 56 78
```

4. Output dari Class Insertion GUI_2511533025.



B. Langkah Penyimpanan

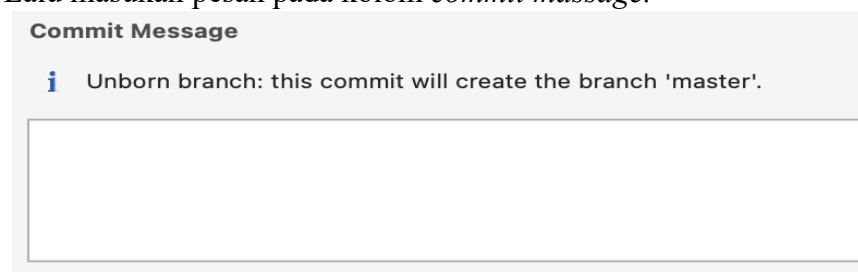
1. Sebelum kita masuk ke langkah, kita harus buat akun github dulu. Jika sudah buat akun github, baru bisa kita simpan di githubnya. Selanjutnya ikuti langkah ini. Tekan kanan mouse pada folder proyek, setelah itu pilih team, terus ke *commit*.



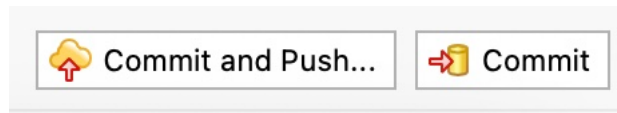
2. Lalu tekan tombol plus 2/double plus warna hijau jika ingin push semua kode, kalau hanya salah satu klik kode yang ingin di push, terus klik tanda plus single.



3. Lalu masukan pesan pada kolom *commit message*.



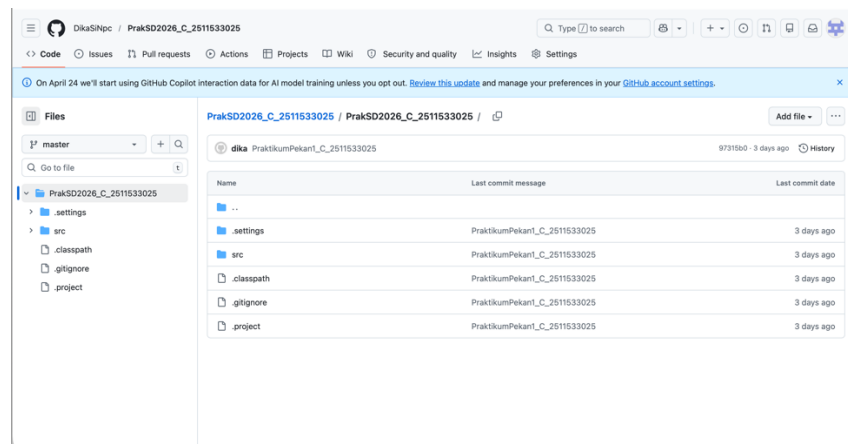
4. Setelah itu tekan *commit* and pus, lalu ikutin arahan untuk memasukan nama dan pw akun git hub kalian, maka projek kalian sudah tersimpan di akun github kalian.



5. Proyek yang sudah disimpan di akun proyek.

 **DikaSiNpc/PrakSD2026_C_2511533025**

6. Gambar proyek apabila sudah masuk di akun github.



Sedikit tambahan jika ingin cek hasil program nya tekan tombol *run* warna hijau.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru atau package yang baru.

BAB 3

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma *Bubble Sort*, *Selection Sort*, dan *Insertion Sort* merupakan teknik pengurutan dasar berbasis perbandingan yang memiliki kompleksitas waktu rata-rata kuadratik, sehingga kurang efisien jika digunakan untuk pengolahan data dalam jumlah yang sangat besar. *Bubble Sort* bekerja dengan cara berulang kali menukar elemen bersebelahan yang tidak sesuai urutannya, *Selection Sort* bekerja dengan mencari elemen minimum atau maksimum dan menempatkannya di posisi yang tepat, sedangkan *Insertion Sort* bekerja dengan menyisipkan elemen ke dalam *sub-array* yang sudah terurut secara iteratif. Meskipun ketiganya sederhana untuk diimplementasikan, *Insertion Sort* terbukti memiliki kinerja yang lebih baik dibandingkan dua algoritma lainnya ketika menangani dataset yang berukuran kecil atau data yang sudah hampir terurut, sementara *Bubble Sort* cenderung paling lambat karena banyaknya operasi penukaran yang dilakukan.

3.2 Saran

Untuk pengembangan selanjutnya, Disarankan agar praktikan tidak hanya berfokus pada keberhasilan kompilasi kode program, melainkan juga melakukan analisis mendalam terhadap jumlah perbandingan dan operasi swap yang terjadi pada setiap algoritma guna memahami perbedaan efisiensi waktu secara nyata. Untuk pengembangan selanjutnya, disarankan untuk menguji program menggunakan variasi dataset yang lebih kompleks, seperti data dengan jumlah ribuan elemen atau data yang urutannya terbalik, serta mencoba menerapkan teknik optimasi sederhana agar dapat melihat peningkatan performa algoritma secara langsung.

DAFTAR PUSTAKA

- Sumber daring (website):

[1] Z. N. Hanilatifa, "Struktur Data: Sorting," Medium, Mar. 25, 2025. <https://zahrahani.medium.com/struktur-data-sorting-6a30fe60684f> (diakses: Mei 20, 2026).

[2] "Algoritma Sort: Pengertian, Jenis, Cara Kerja, dan Contoh Lengkap," UNESA, D4 Manajemen Informatika, Nov. 22, 2025. <https://terapan-ti.vokasi.unesa.ac.id/post/algoritma-sort-pengertian-jenis-cara-kerja-dan-contoh-lengkap> (diakses: Mei 20, 2026).

LAPORAN TUGAS PRAKTIKUM PEKAN 7

1. Intruksi Tugas Praktikum Pekan 7

Topik : Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort)

Tema : Pengurutan Nama Mahasiswa Secara Alfabetis Menggunakan GUI

1. Deskripsi Program

Program dikembangkan menggunakan Java GUI (Swing) untuk menampilkan:

- proses pengurutan data mahasiswa secara interaktif.
- Program menyediakan beberapa fitur utama, yaitu:
- input data mahasiswa secara manual melalui form GUI.
- Menyimpan data mahasiswa ke dalam array atau ArrayList object.
- Pemilihan algoritma sorting menggunakan ComboBox.
- Menampilkan visualisasi proses sorting langkah demi langkah.
- Menampilkan hasil akhir pengurutan nama mahasiswa secara ascending (A-Z).
- Perbandingan string dilakukan menggunakan method `compareToIgnoreCase()` agar proses sorting tidak membedakan huruf kapital dan huruf kecil.

2. Struktur Program

Program terdiri dari beberapa bagian utama:

A. Kelas ADT Mahasiswa

Class digunakan untuk menyimpan data mahasiswa.

Atribut:

- Nama Mahasiswa (String), NIM Mahasiswa (String), dan Program Studi (String).

Method:

- Constructor
- Getter dan Setter
- `toString()`

B. Kelas GUI Utama

Class GUI digunakan untuk:

- Menampilkan form input data.
- Menampilkan tabel data mahasiswa.
- Menampilkan ComboBox pilihan algoritma sorting.
- Menampilkan proses visualisasi sorting.
- Menjalankan algoritma sorting sesuai pilihan pengguna.

C. Method Sorting

Program menyediakan tiga method sorting terpisah:

- insertionSort()
- selectionSort()
- bubbleSort()

3. Komponen GUI

GUI program terdiri dari beberapa komponen berikut:

- JTextField, Digunakan untuk input nama mahasiswa, NIM, dan program studi.
- JButton, Digunakan untuk: Menambahkan data, Menghapus data, dan Memulai sorting.
- JComboBox, Digunakan untuk memilih algoritma sorting: Insertion Sort, Selection Sort, dan Bubble Sort.
- JTable / JTextArea, Digunakan untuk menampilkan: Data mahasiswa, Langkah-langkah proses sorting, dan Hasil akhir sorting.

4. Alur Program

1. Pengguna menginput data mahasiswa melalui GUI.
2. Data disimpan ke dalam array atau ArrayList.
3. Pengguna memilih algoritma sorting pada ComboBox.
4. Pengguna menekan tombol “Mulai Sorting”.
5. Program menjalankan algoritma sorting yang dipilih.
6. Proses sorting ditampilkan visualisasinya langkah demi langkah.
7. Hasil akhir pengurutan ditampilkan pada GUI.

2. Kode program Class Mahasiswa_2511533025

```
package TugasPekan7_2511533025;

class Mahasiswa_2511533025 {
    private String nama_3025;
    private String nim_3025;
    private String prodi_3025;
    public Mahasiswa_2511533025(String nama_3025,
String nim_3025, String prodi_3025) {
        this.nama_3025 = nama_3025;
        this.nim_3025 = nim_3025;
        this.prodi_3025 = prodi_3025;
    }
    public String getNama() {
        return nama_3025;
    }
    public String getNim() {
        return nim_3025;
    }
    public String getProdi() {
        return prodi_3025;
    }
    @Override
```

```

        public String toString() {
            return nama_3025 + " - " + nim_3025 + " - " +
prodi_3025;
        }
    }
}

```

3. Kode program Class SortingMahasiswaGUI _2511533025

```

package TugasPekan7_2511533025;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.ArrayList;

public class SortingMahasiswaGUI_2511533025 extends
JFrame {

    private JTextField tfNama_3025, tfNim_3025,
tfProdi_3025;
    private JButton btnTambah_3025, btnSorting_3025,
btnReset_3025;
    private JComboBox<String> cbSorting_3025;
    private JTextArea areaProses_3025;
    private JTable table_3025;
    private DefaultTableModel model_3025;
    private ArrayList<Mahasiswa_2511533025> data_3025
= new ArrayList<>();
    private ArrayList<Mahasiswa_2511533025>
backupData_3025 = new ArrayList<>();

    public SortingMahasiswaGUI_2511533025() {
        setTitle("Sorting Mahasiswa");
        setSize(800, 550);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        // =====
        // PANEL INPUT
        // =====

        JPanel panelInput_3025 = new JPanel(new
GridLayout(4, 2, 5, 5));

        panelInput_3025.add(new JLabel("Nama
Mahasiswa"));
        tfNama_3025 = new JTextField();
        panelInput_3025.add(tfNama_3025);

        panelInput_3025.add(new JLabel("NIM"));
        tfNim_3025 = new JTextField();
        panelInput_3025.add(tfNim_3025);
    }
}

```

```

        panelInput_3025.add(new JLabel("Program
Studi"));
        tfProdi_3025 = new JTextField();
        panelInput_3025.add(tfProdi_3025);

        btnTambah_3025 = new JButton("Tambah Data");
        panelInput_3025.add(btnTambah_3025);

        cbSorting_3025 = new JComboBox<>(new
String[]{
            "Insertion Sort",
            "Selection Sort",
            "Bubble Sort"
        });
        panelInput_3025.add(cbSorting_3025);
        add(panelInput_3025, BorderLayout.NORTH);

        // =====
        // TABEL
        // =====

        model_3025 = new DefaultTableModel();
        model_3025.addColumn("Nama");
        model_3025.addColumn("NIM");
        model_3025.addColumn("Program Studi");
        table_3025 = new JTable(model_3025);

        JScrollPane scrollTable_3025 = new
JScrollPane(table_3025);
        add(scrollTable_3025, BorderLayout.CENTER);

        // =====
        // AREA PROSES SORTING
        // =====

        areaProses_3025 = new JTextArea();
        areaProses_3025.setFont(new
Font("Monospaced", Font.PLAIN, 14));

        JScrollPane scrollArea_3025 = new
JScrollPane(areaProses_3025);

        // =====
        // BUTTON
        // =====

        btnSorting_3025 = new JButton("Mulai
Sorting");
        btnReset_3025 = new JButton("Reset");

        JPanel panelButton_3025 = new JPanel();
        panelButton_3025.add(btnSorting_3025);
        panelButton_3025.add(btnReset_3025);

```

```

        JPanel panelBawah_3025 = new JPanel(new
        BorderLayout());
        panelBawah_3025.add(panelButton_3025,
        BorderLayout.NORTH);
        panelBawah_3025.add(scrollArea_3025,
        BorderLayout.CENTER);
        add(panelBawah_3025, BorderLayout.SOUTH);

        // =====
        // EVENT TAMBAH DATA
        // =====

        btnTambah_3025.addActionListener(e ->
        tambahData());

        // =====
        // EVENT SORTING
        // =====

        btnSorting_3025.addActionListener(e ->
        prosesSorting());

        // =====
        // EVENT RESET
        // =====

        btnReset_3025.addActionListener(e ->
        resetData());
    }
    // =====
    // METHOD TAMBAH DATA
    // =====

    private void tambahData() {

        String nama_3025 = tfNama_3025.getText();
        String nim_3025 = tfNim_3025.getText();
        String prodi_3025 = tfProdi_3025.getText();
        if (nama_3025.isEmpty() || nim_3025.isEmpty()
        || prodi_3025.isEmpty()) {
            JOptionPane.showMessageDialog(
                this,
                "Semua data harus diisi!"
            );
            return;
        }
        Mahasiswa_2511533025 mhs_3025 = new
        Mahasiswa_2511533025(nama_3025, nim_3025,
        prodi_3025);
        data_3025.add(mhs_3025);
        model_3025.addRow(new Object[]{
            nama_3025,
            nim_3025,
            prodi_3025
        });
    }

```

```

    });
    tfNama_3025.setText("");
    tfNim_3025.setText("");
    tfProdi_3025.setText("");
}

// =====
// METHOD SORTING
// =====

private void prosesSorting() {

    areaProses_3025.setText("");
    backupData_3025.clear();
    for (Mahasiswa_2511533025 m_3025 : data_3025)
    {
        backupData_3025.add(
            new Mahasiswa_2511533025(
                m_3025.getNama(),
                m_3025.getNim(),
                m_3025.getProdi()
            )
        );
    }
    String pilihan_3025 =
    cbSorting_3025.getSelectedItem().toString();
    if (pilihan_3025.equals("Insertion Sort")) {
        insertionSort();
    } else if (pilihan_3025.equals("Selection
Sort")) {
        selectionSort();
    } else {
        bubbleSort();
    }
    refreshTable();
}

// =====
// INSERTION SORT
// =====

private void insertionSort() {

    areaProses_3025.append("=== INSERTION SORT
===\n\n");
    for (int i_3025 = 1; i_3025 <
data_3025.size(); i_3025++) {
        Mahasiswa_2511533025 key_3025 =
data_3025.get(i_3025);
        int j_3025 = i_3025 - 1;
        while (j_3025 >= 0 &&
            data_3025.get(j_3025).getNama()
.compareToIgnoreCase(key_3025.getNama()) > 0) {

```

```

        data_3025.set(j_3025 + 1,
data_3025.get(j_3025));
        j_3025--;
    }
    data_3025.set(j_3025 + 1, key_3025);
    tampilkanLangkah(i_3025);
}
}

// =====
// SELECTION SORT
// =====

private void selectionSort() {

    areaProses_3025.append("=== SELECTION SORT
===\n\n");
    for (int i_3025 = 0; i_3025 <
data_3025.size() - 1; i_3025++) {
        int min_3025 = i_3025;
        for (int j_3025 = i_3025 + 1; j_3025 <
data_3025.size(); j_3025++) {
            if (data_3025.get(j_3025).getNama()
                .compareToIgnoreCase(

data_3025.get(min_3025).getNama()) < 0) {
                min_3025 = j_3025;
            }
        }
        Mahasiswa_2511533025 temp_3025 =
data_3025.get(i_3025);
        data_3025.set(i_3025,
data_3025.get(min_3025));
        data_3025.set(min_3025, temp_3025);
        tampilkanLangkah(i_3025 + 1);
    }
}

// =====
// BUBBLE SORT
// =====

private void bubbleSort() {

    areaProses_3025.append("=== BUBBLE SORT
===\n\n");
    for (int i_3025 = 0; i_3025 <
data_3025.size() - 1; i_3025++) {
        for (int j_3025 = 0; j_3025 <
data_3025.size() - i_3025 - 1; j_3025++) {
            if (data_3025.get(j_3025).getNama()
                .compareToIgnoreCase(
                    data_3025.get(j_3025
+ 1).getNama()) > 0) {

```

```

        Mahasiswa_2511533025 temp_3025 =
data_3025.get(j_3025);
        data_3025.set(j_3025,
data_3025.get(j_3025 + 1));
        data_3025.set(j_3025 + 1,
temp_3025);
    }
    }
    tampilkanLangkah(i_3025 + 1);
}
}

// =====
// MENAMPILKAN LANGKAH
// =====

private void tampilkanLangkah(int langkah) {
    areaProses_3025.append("Langkah " + langkah +
" : ");
    for (Mahasiswa_2511533025 m_3025 : data_3025)
    {
        areaProses_3025.append(m_3025.getNama() +
" ");
    }
    areaProses_3025.append("\n");
}

// =====
// REFRESH TABEL
// =====

private void refreshTable() {
    model_3025.setRowCount(0);
    for (Mahasiswa_2511533025 m_3025 : data_3025)
    {
        model_3025.addRow(new Object[]{
            m_3025.getNama(),
            m_3025.getNim(),
            m_3025.getProdi()
        });
    }
}

// =====
// RESET DATA DAN JSCROLLPANE
// =====

private void resetData() {
    // KEMBALIKAN DATA AWAL
    data_3025.clear();
}

```

```

        for (Mahasiswa_2511533025 m_3025 :
backupData_3025) {
            data_3025.add(
                new Mahasiswa_2511533025(
                    m_3025.getNama(),
                    m_3025.getNim(),
                    m_3025.getProdi()
                )
            );
        }
        refreshTable();
        // BERSIHKAN TEXT AREA PADA JSCROLLPANE
        areaProses_3025.setText("");
    }

    // =====
    // MAIN
    // =====

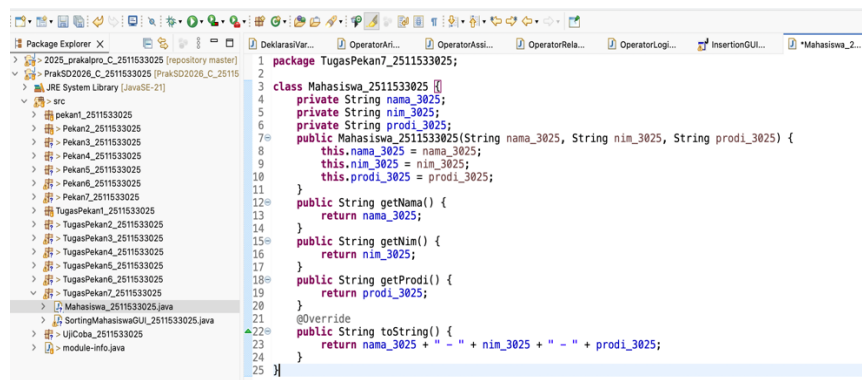
    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {
            new
            SortingMahasiswaGUI_2511533025().setVisible(true);
        });
    }
}

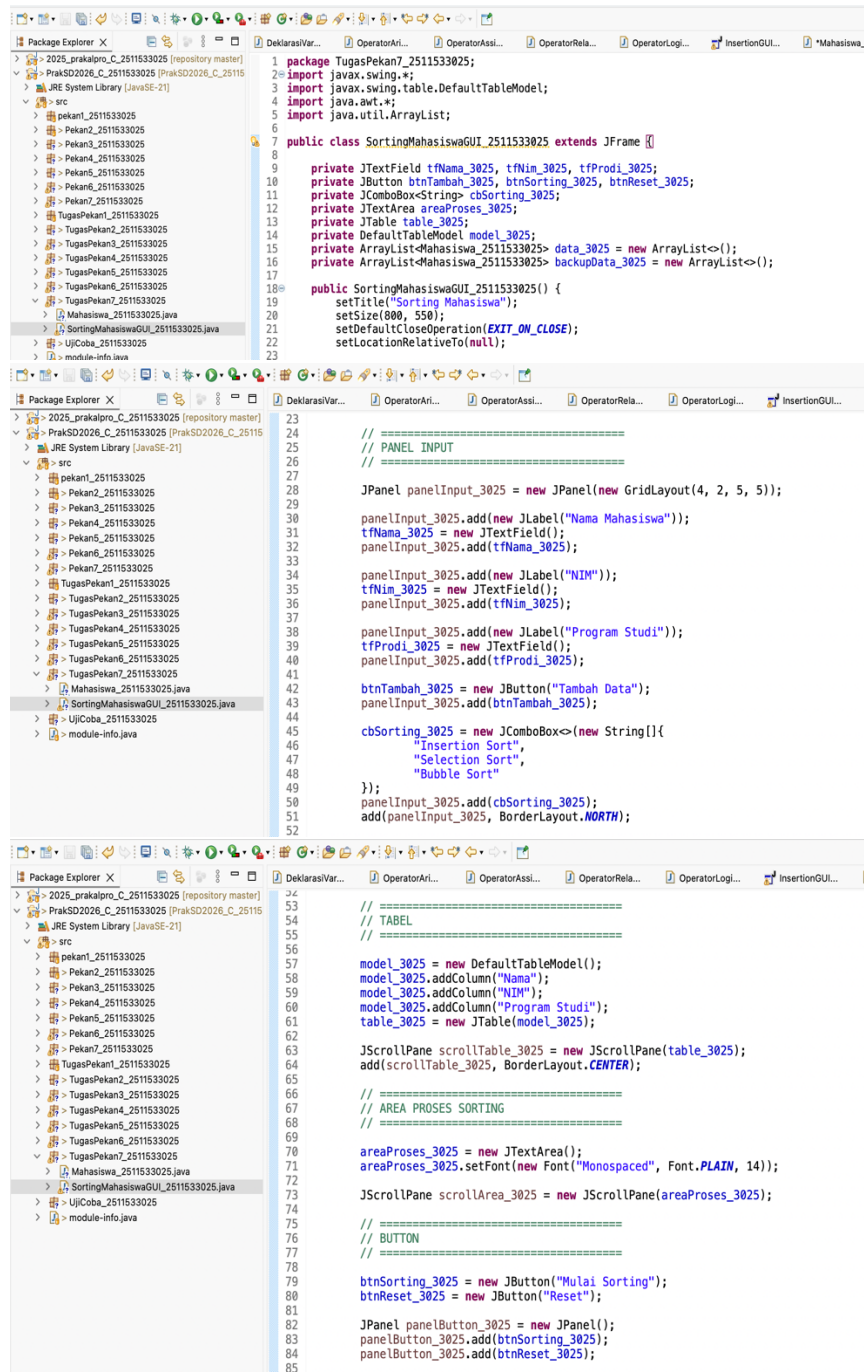
```

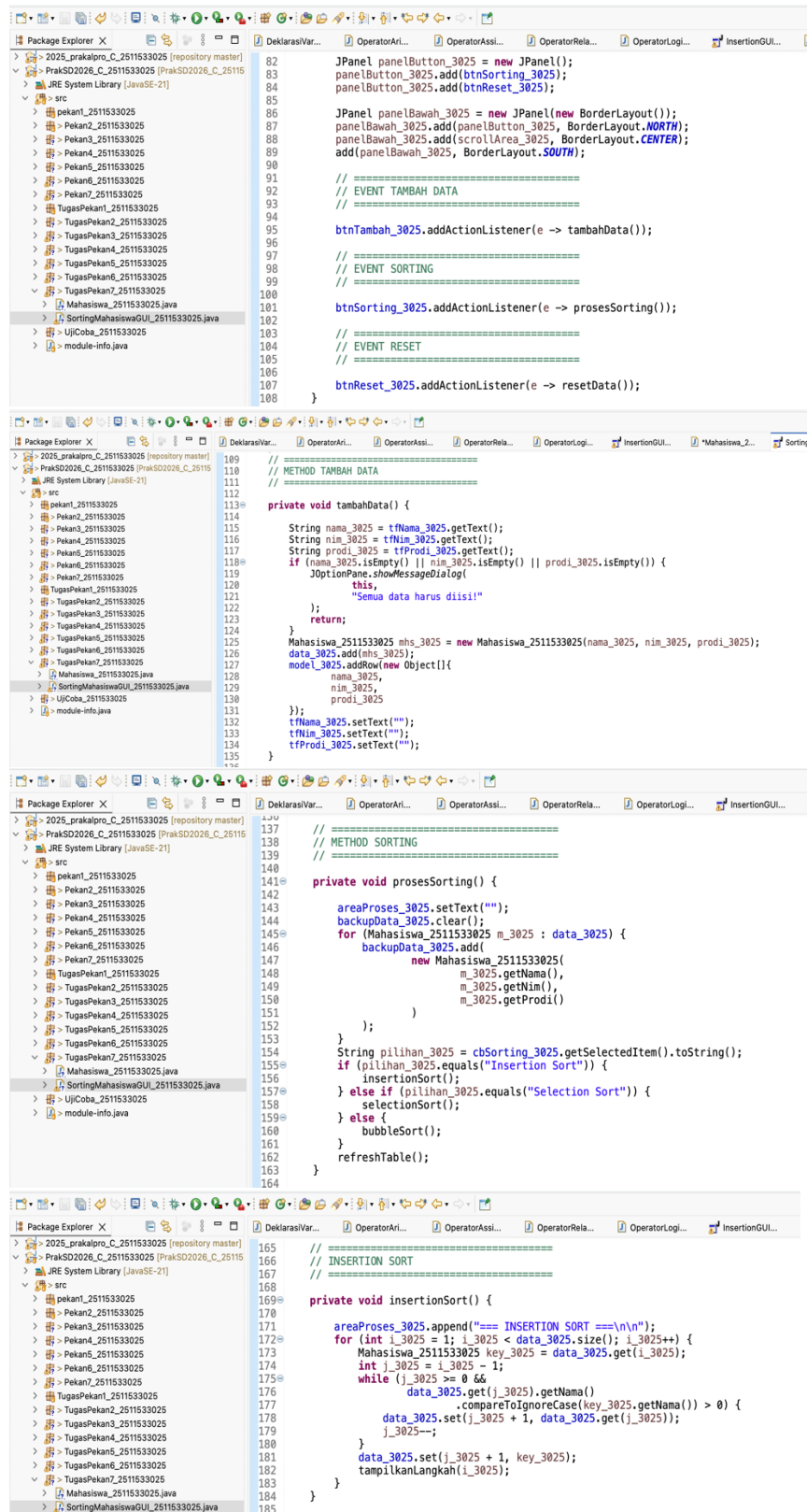
4. Gambar kode program dan output program.

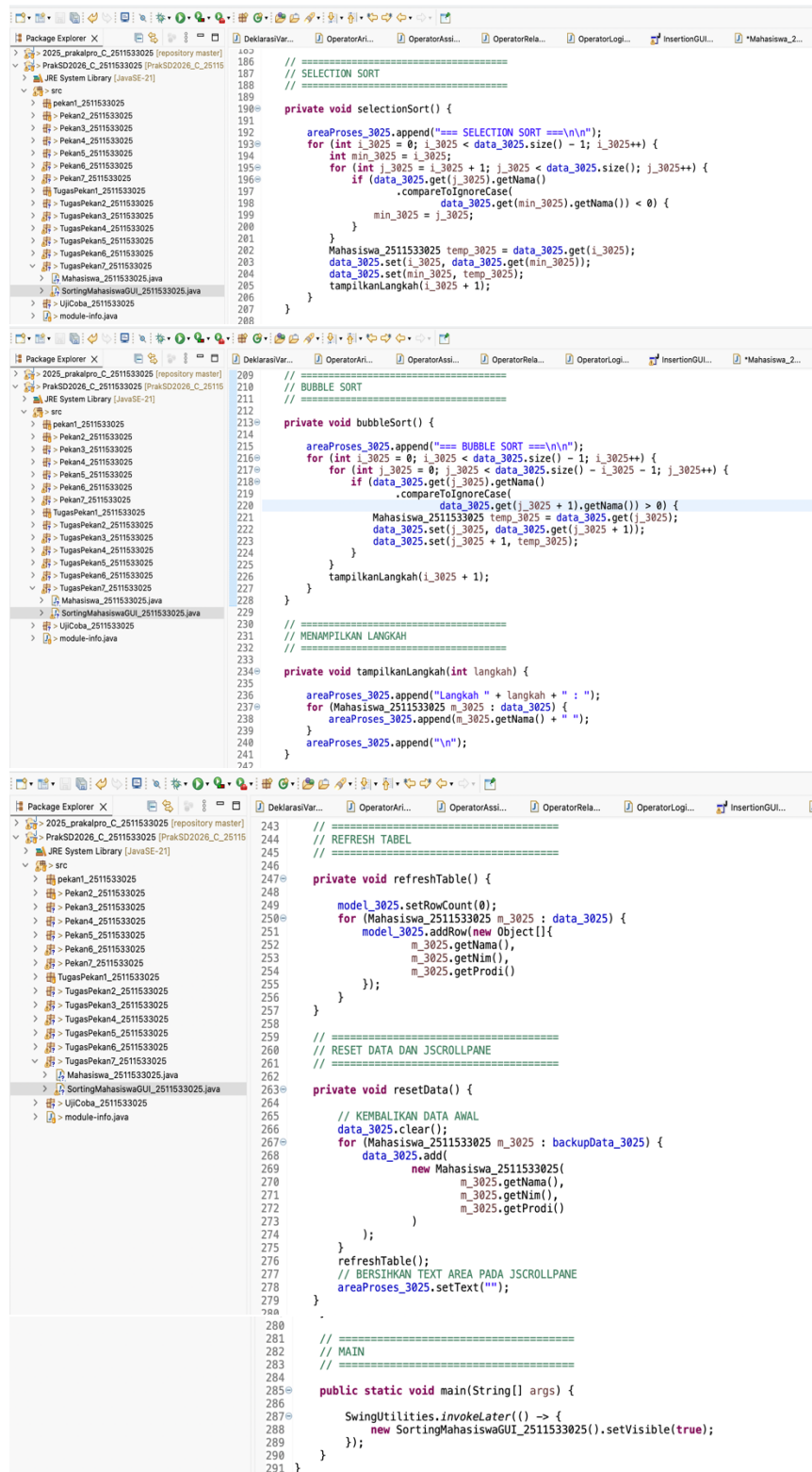
1. Gambar Class Mahasiswa_2511533025



2. Gambar Class SortingMahasiswaGUI_2511533025







3. Gambar output code program

Sorting Mahasiswa

Nama Mahasiswa

NIM

Program Studi

Tambah Data

Insertion Sort

Nama	NIM	Program Studi
Arkan	2511533015	Informatika
Atar	2511533031	Informatika
Dika	2511533025	Informatika
Fajar	2511533025	Informatika
Sofian	2511533029	Informatika

Mulai Sorting

Reset

=== INSERTION SORT ===

Langkah 1 : Dika Fajar Arkan Atar Sofian
 Langkah 2 : Arkan Dika Fajar Atar Sofian
 Langkah 3 : Arkan Atar Dika Fajar Sofian
 Langkah 4 : Arkan Atar Dika Fajar Sofian

Sorting Mahasiswa

Nama Mahasiswa

NIM

Program Studi

Tambah Data

Selection Sort

Nama	NIM	Program Studi
Arkan	2511533015	Informatika
Atar	2511533031	Informatika
Dika	2511533025	Informatika
Fajar	2511533025	Informatika
Sofian	2511533029	Informatika

Mulai Sorting

Reset

=== SELECTION SORT ===

Langkah 1 : Arkan Fajar Dika Atar Sofian
 Langkah 2 : Arkan Atar Dika Fajar Sofian
 Langkah 3 : Arkan Atar Dika Fajar Sofian
 Langkah 4 : Arkan Atar Dika Fajar Sofian

Sorting Mahasiswa

Nama Mahasiswa

NIM

Program Studi

Tambah Data

Bubble Sort

Nama	NIM	Program Studi
Arkan	2511533015	Informatika
Atar	2511533031	Informatika
Dika	2511533025	Informatika
Fajar	2511533025	Informatika
Sofian	2511533029	Informatika

=== BUBBLE SORT ===

Langkah 1 : Dika Arkan Atar Fajar Sofian
 Langkah 2 : Arkan Atar Dika Fajar Sofian
 Langkah 3 : Arkan Atar Dika Fajar Sofian
 Langkah 4 : Arkan Atar Dika Fajar Sofian

Sorting Mahasiswa

Nama Mahasiswa

NIM

Program Studi

Tambah Data

Bubble Sort

Nama	NIM	Program Studi
Dika	2511533025	Informatika
Fajar	2511533025	Informatika
Arkan	2511533015	Informatika
Atar	2511533031	Informatika
Sofian	2511533029	Informatika

5. Penjelasan sederhana tentang kode program

Program di atas merupakan aplikasi Java GUI menggunakan Swing yang digunakan untuk mengurutkan data mahasiswa berdasarkan nama

secara alfabetis. Kelas Mahasiswa digunakan sebagai ADT (Abstract Data Type) untuk menyimpan data mahasiswa. Pada kelas ini terdapat tiga atribut utama yaitu nama, nim, dan prodi yang bertipe String. Kelas ini memiliki constructor yang digunakan untuk mengisi data saat objek dibuat, serta beberapa method getter seperti getName(), getNim(), dan getProdi() untuk mengambil nilai dari setiap atribut. Selain itu terdapat method toString() yang berfungsi menampilkan data mahasiswa dalam bentuk teks. Dengan adanya kelas ini, data mahasiswa dapat disimpan dan dikelola dengan lebih terstruktur dalam program sorting.

Program memiliki form input yang terdiri dari nama mahasiswa, NIM, dan program studi menggunakan JTextField. Data yang diinput pengguna akan disimpan ke dalam ArrayList bertipe objek Mahasiswa, kemudian ditampilkan pada JTable. Program juga menyediakan JComboBox untuk memilih algoritma sorting, yaitu Insertion Sort, Selection Sort, dan Bubble Sort. Saat tombol "Mulai Sorting" ditekan, program akan menjalankan algoritma yang dipilih dan membandingkan nama mahasiswa menggunakan compareToIgnoreCase() agar huruf besar dan kecil dianggap sama. Proses pengurutan ditampilkan langkah demi langkah pada JTextArea yang berada di dalam JScrollPane, sehingga pengguna dapat melihat visualisasi proses sorting secara interaktif. Selain itu, terdapat tombol reset yang berfungsi untuk mengembalikan data ke kondisi awal sekaligus membersihkan hasil visualisasi sorting pada area proses tanpa perlu menjalankan ulang program.